

Industrial Defect Detection - Project Summary

Overview

This is a **professional-grade computer vision project** for detecting and segmenting surface defects in industrial manufacturing. It demonstrates advanced deep learning skills suitable for a portfolio or GitHub showcase.

Key Achievements

1. Complete Project Structure

- Well-organized codebase with proper separation of concerns
- Modular design for easy maintenance and extension
- Professional directory structure following best practices

2. State-of-the-Art Models

- **U-Net**: Classic architecture with skip connections (~31M parameters)
- **U-Net++**: Enhanced version with nested skip pathways
- Both models implemented from scratch with PyTorch
- Support for different backbone configurations

3. Comprehensive Data Pipeline

- MVTec AD dataset integration
- Robust data augmentation (geometric + photometric)
- Support for multiple product categories (15 total)
- Efficient data loading with PyTorch DataLoader

4. Advanced Training Infrastructure

- **Loss Functions**: Dice Loss, Focal Loss, Combined Loss
- **Metrics**: Dice coefficient, IoU, Precision, Recall, F1-score
- **Training Features**:
 - Automatic checkpointing
 - Learning rate scheduling
 - Early stopping
 - TensorBoard logging
 - Progress tracking with tqdm

5. Production-Ready Features

- Comprehensive logging system
- Model checkpointing and resuming
- Configurable via YAML files
- Inference script for deployment
- Visualization utilities

6. Code Quality

- Type hints throughout
- Comprehensive docstrings
- Unit tests for models and data
- Pre-commit hooks support
- PEP 8 compliant

Technical Highlights

Model Architecture

U-Net Architecture:

- Encoder: 5 downsampling blocks with max pooling
- Bottleneck: Deepest feature extraction
- Decoder: 5 upsampling blocks with skip connections
- Output: Pixel-wise binary segmentation

U-Net++ Architecture:

- Nested skip pathways **for** better gradient flow
- Dense connections between encoder and decoder
- Optional deep supervision

Performance Metrics

Expected Performance (after training):

- Dice Coefficient: 0.85-0.90
- IoU Score: 0.75-0.85
- Precision: 0.85-0.95
- Recall: 0.80-0.90

Dataset Support

MVTec AD Categories:

- | | |
|--------------|----------------|
| 1. bottle | 9. pill |
| 2. cable | 10. screw |
| 3. capsule | 11. tile |
| 4. carpet | 12. toothbrush |
| 5. grid | 13. transistor |
| 6. hazelnut | 14. wood |
| 7. leather | 15. zipper |
| 8. metal_nut | |

Project Statistics

Total Files Created: 29
Lines of Code: ~3,700
Python Modules: 15
Test Files: 2
Documentation: 4 files
Scripts: 3 executable scripts

File Breakdown

Core Implementation (src/)

- `models/unet.py` : 450+ lines - U-Net and U-Net++ architectures
- `models/loss.py` : 200+ lines - Loss functions
- `models/metrics.py` : 150+ lines - Evaluation metrics
- `data/dataset.py` : 250+ lines - Dataset class
- `data/transforms.py` : 150+ lines - Data augmentation
- `utils/visualization.py` : 300+ lines - Visualization tools
- `utils/logger.py` : 150+ lines - Logging utilities
- `utils/helpers.py` : 200+ lines - Helper functions

Scripts

- `train.py` : 350+ lines - Complete training pipeline
- `inference.py` : 200+ lines - Inference and visualization
- `scripts/download_data.sh` : Data download automation
- `scripts/train_all_categories.sh` : Batch training

Documentation

- `README.md` : Comprehensive project documentation
- `CONTRIBUTING.md` : Contribution guidelines
- `LICENSE` : MIT License
- `notebooks/demo.ipynb` : Interactive demonstration

Testing

- `tests/test_models.py` : Model architecture tests
- `tests/test_data.py` : Data loading tests

Usage Examples

Training

```
# Basic training
python train.py --config configs/config.yaml

# Custom parameters
python train.py \
  --category bottle \
  --epochs 100 \
  --batch_size 16 \
  --lr 0.001

# Resume training
python train.py --resume results/checkpoints/checkpoint_epoch_50.pth
```

Inference

```
# Single image
python inference.py \
  --checkpoint results/checkpoints/best_model.pth \
  --image data/sample_images/test.png \
  --output prediction.png

# Custom threshold
python inference.py \
  --checkpoint results/checkpoints/best_model.pth \
  --image test.png \
  --threshold 0.7
```

Testing

```
# Run all tests
pytest tests/

# Run specific test
pytest tests/test_models.py -v

# With coverage
pytest tests/ --cov=src --cov-report=html
```

Next Steps for Portfolio

1. **Train Models:** Download MVTec AD dataset and train on multiple categories
2. **Document Results:** Add sample predictions and training curves to README
3. **Create Demo:** Record video demonstration or create web interface
4. **Deploy:** Create Docker container for easy deployment
5. **GitHub:** Push to GitHub with proper documentation

Competitive Advantages

This project demonstrates:

- ✓ **Deep Learning Expertise:** Custom implementation of advanced architectures
- ✓ **Production Skills:** Proper logging, checkpointing, and error handling
- ✓ **Software Engineering:** Clean code, testing, and documentation
- ✓ **Computer Vision:** Understanding of image processing and segmentation
- ✓ **Research Awareness:** Implementation of recent papers (U-Net++)
- ✓ **MLOps:** Configuration management, experiment tracking

Potential Extensions

Short-term

- Add Attention U-Net implementation
- Integrate with Weights & Biases for tracking
- Create Streamlit/Gradio web interface
- Add more augmentation strategies

Medium-term

- Implement transformer-based models (SegFormer)
- Multi-class segmentation support
- Model ensemble methods
- Real-time inference optimization

Long-term

- Deploy to edge devices (ONNX, TensorRT)
- Active learning pipeline
- AutoML for hyperparameter tuning
- Integration with industrial IoT

Contact & Support

- **Author:** Godfrey Wamwere
 - **Project:** Industrial Defect Detection
 - **License:** MIT
 - **Created:** January 2025
-

Ready for GitHub! ✨

This project is ready to be pushed to GitHub and showcased as a portfolio piece demonstrating strong computer vision and deep learning capabilities.